

Chapter 14

Disciplined Convex Programming

Algorithms for Optimization, 2nd ed. (2026)
Kochenderfer & Wheeler

Lecture Slides

Chapter Outline

- 1 Canonical Form of a DCP
- 2 Verification of Convexity
- 3 Canonicalization
- 4 Solving with CVXPY
- 5 Summary & Exercises

What is Disciplined Convex Programming?

Motivation

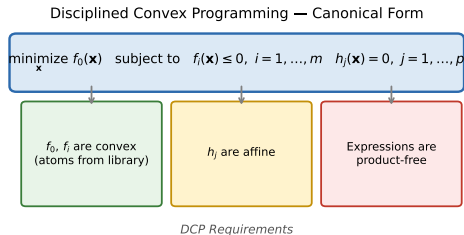
Convex optimization problems are those with a **convex objective** and **convex/affine constraints**.

General techniques exist to find global optima, but:

- Verifying convexity by hand is tedious and error-prone
- We need *automated* methods

DCP Definition

A **disciplined convex program (DCP)** is a convex optimization problem written in such a way that *automated methods can verify its convexity* and transcribe it into canonical form for any standard solver.



Canonical Form of a DCP

Standard Convex Problem (Eq. 14.1)

$$\underset{\mathbf{x}}{\text{minimize}} \quad f_0(\mathbf{x}) \quad \text{subject to} \quad \begin{cases} f_i(\mathbf{x}) \leq 0 & i = 1, \dots, m \\ h_j(\mathbf{x}) = 0 & j = 1, \dots, p \end{cases}$$

where $\mathbf{x} \in \mathbb{R}^n$, f_0, f_i are **convex**, and h_j are **affine**.

DCP Requirements

- ① **Product-free expressions** — no products of non-constant sub-expressions
- ② **Top-level production rules** — objective must be convex; inequality LHS convex, RHS concave
- ③ **Sign rules** — track the sign/domain of each sub-expression
- ④ **Composition rules** — curvature propagates through function composition

Running Example (Ex. 14.1)

Problem

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{Ax} - \mathbf{b}\|_1 \quad \text{s.t.} \quad \|\mathbf{Cx} - \mathbf{d}\|_{1.5} \leq 3$$

with

$$\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ -5 \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} 0 & 1 \\ 3 & -4 \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

DCP Status

- $\|\cdot\|_1$ is **convex**, applied to affine expression $\Rightarrow$ convex
- $\|\cdot\|_{1.5}$ is **convex**, applied to affine expression $\Rightarrow$ convex
- Constraint: convex $\leq$ constant $\checkmark$
- Objective: minimize convex $\checkmark$

This is a valid DCP!

Key insight

Each function comes from an *atom library* with known curvature and monotonicity properties.

Atom Library

Atom Library — Common Functions

Function	Domain	Curvature	Monotonicity	Range
$ x $	$\mathbb{R}$	convex	nonmonotone	$[0, \infty)$
x^2	$\mathbb{R}$	convex	nonmonotone	$[0, \infty)$
$\max(x_1, x_2)$	$\mathbb{R}^2$	convex	nondecreasing	$\mathbb{R}$
$\exp(x)$	$\mathbb{R}$	convex	nondecreasing	$(0, \infty)$
$\log(x)$	$(0, \infty)$	concave	nondecreasing	$\mathbb{R}$
$\sqrt{x}$	$[0, \infty)$	concave	nondecreasing	$[0, \infty)$
$\min(x_1, x_2)$	$\mathbb{R}^2$	concave	nondecreasing	$\mathbb{R}$
$\ \mathbf{x}\ _p$	$\mathbb{R}^n$	convex	nonmonotone	$[0, \infty)$

What an Atom Provides

Each atom f in the library specifies:

- **Domain** — where f is defined
- **Curvature** — convex / concave / affine
- **Monotonicity** — nondecreasing / nonincreasing (per argument)
- **Range** — bounds on f 's output

Sets for Atoms

Also defined as atoms:

- $\mathbb{R}^n$ — real numbers
- Positive orthant $\mathbb{R}_+^n$
- Epigraphs/hypographs of functions

Verification — Overview

Goal

Automatically verify that a given problem is a valid DCP.

Two-Stage Verification

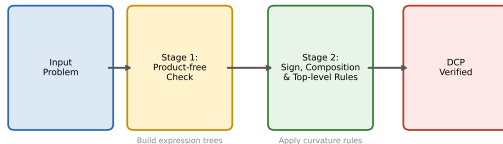
1 Stage 1 — Product-free check

All expressions must be product-free
(no products of non-constant
sub-expressions)

2 Stage 2 — Rule checking

Verify top-level, sign, and composition rules
hold throughout the expression tree

Two-Stage Automatic DCP Verification



Product-Free Expressions

Definition (Eq. 14.5)

An expression e is **product-free** if it can be written as:

$$e = a + \mathbf{b}^\top \mathbf{x} + \sum_{j=1}^q c_j f_j(\mathbf{e}_j)$$

where $a \in \mathbb{R}$, $\mathbf{b} \in \mathbb{R}^n$, each f_j is from the atom library, and each $\mathbf{e}_j$ is a vector of product-free sub-expressions.

Product-free (valid)

- $3 + 2x_1 + \log(x_2)$
- $\max(x_1^2 - 3, x_2)$
- $\|\mathbf{Ax} - \mathbf{b}\|_2$

Not product-free (invalid)

- $x_1 \cdot x_2$ (product of variables)
- $x_1^2 = x_1 \cdot x_1$ (also invalid as literal product)
- $\sin(x_1) \cdot \log(x_2)$

Note

Sign Rules

Scaling a convex expression

If $f(\mathbf{x})$ is convex and $c > 0$:

- $c \cdot f(\mathbf{x})$ is **convex**
- $-c \cdot f(\mathbf{x})$ is **concave**
- $c \cdot f(\mathbf{x})$ where $c < 0$: **concave**

Sum Rules

- convex + convex = **convex**
- concave + concave = **concave**
- affine + anything = **same curvature**
- convex + concave = **neither**
- convex $\times$ positive const = **convex**
- convex $\times$ negative const = **concave**

Example: $3 + 2 \log(1 + x_1) - \sin(x_2)$ for $x_1 \geq 0, x_2 \in [-\pi, 0]$

- $\log(1 + x_1)$ is concave $\Rightarrow 2 \log(1 + x_1)$ is concave
- $\sin(x_2)$ for $x_2 \in [-\pi, 0]$ is convex $\Rightarrow -\sin(x_2)$ is concave

Composition Rules

Key Theorem (Eqs. 14.3–14.4)

For $f(\varepsilon_1, \dots, \varepsilon_m)$, if f is expected to be **convex**:

- f is affine or convex; and for each argument ε_i :
 - f is *nondecreasing* in ε_i over $\mathcal{R}$, and ε_i is convex; **or**
 - f is *nonincreasing* in ε_i over $\mathcal{R}$, and ε_i is concave; **or**
 - ε_i is affine

Mirror rules apply for concave f .

Decomposition Consequence (Eq. 14.7)

If f is convex and nondecreasing:

$$f(g(\mathbf{x})) \leq y \quad \Longleftrightarrow \quad \begin{cases} f(z) \leq y \\ g(\mathbf{x}) \leq z \end{cases}$$

This enables *efficient decomposition* into sub-problems.

Composition Rule — Examples

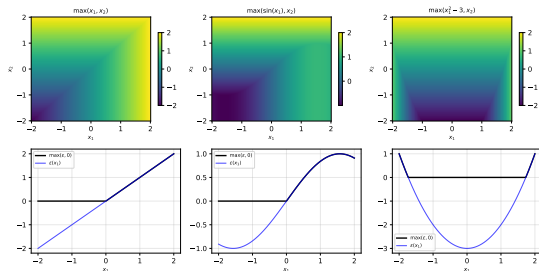
Ex. 14.3: $f(g(\mathbf{x})) = 1/x$ where
 $g = 3 + 2 \log(1 + x_1) - \sin x_2$

- $f(x) = 1/x$ is *convex* and *nonincreasing* for $x > 0$
- Need: g is *concave* and $g \in [3, \infty)$ (where f is nonincreasing)
- Compute range:

$$\begin{aligned} \text{range}(g) &\subseteq 3 + 2 \cdot \text{range}(\log(1 + x_1)) - \sin x_2 \\ &= [3, 3] + [0, \infty) + [0, 1] = [3, \infty) \end{aligned}$$

- g is concave (shown earlier), $\text{range} \subseteq [3, \infty)$
- $\Rightarrow f(g(\mathbf{x}))$ is **convex**

Composition Rules: $\max(\varepsilon(x_1), x_2)$ for Various ε



Ex. 14.5: $\max(\varepsilon(x_1), x_2)$

- $\varepsilon = x_1$: both affine $\Rightarrow$ **convex**
- $\varepsilon = \sin(x_1)$: neither $\Rightarrow$ **not convex**
- $\varepsilon = x_1^2 - 3$: convex $\Rightarrow$ **convex**
- $\varepsilon = \log(x_1) - 1$: concave $\Rightarrow$ **not convex**

Top-Level Production Rules

Rules for the Whole Problem

Context	Required curvature	Example
Minimize f_0	f_0 must be convex	$\min \ \mathbf{x}\ _2$
Maximize f_0	f_0 must be concave	$\max \log(\det X)$
Inequality LHS	must be convex	$g(\mathbf{x}) \leq 0$
Inequality RHS	must be concave	$0 \leq h(\mathbf{x})$
Equality LHS/RHS	must be affine	$\mathbf{Ax} = \mathbf{b}$

Why affine equality?

An equality $f(\mathbf{x}) = g(\mathbf{x})$ is both an upper *and* lower bound, so it requires f to be *both* convex and concave — i.e. affine.

Automatic Verification — Expression Tree (Ex. 14.6)

Problem

$$\underset{x_1, x_2}{\text{minimize}} \quad \text{pow}(a_1 x_1 + a_2 x_2 - b_1, 2)$$

$$\text{s.t.} \quad c_1 x_1 + c_2 x_2 = d, \quad \sqrt{\min(x_1, x_2)} \geq 1 - \log(x_2)$$

Verification Tree (simplified)

minimize

↪ $\text{pow}(\varepsilon, 2)$ convex

↪ $-b_1 + a_1 x_1 + a_2 x_2$ affine

subject to

↪ $=$: affine=affine ✓

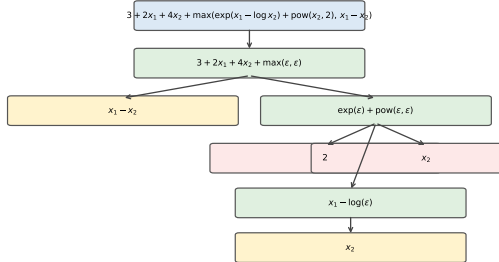
↪ $\geq$: concave $\geq$ convex ✓

↪ $\sqrt{\varepsilon}$ concave, nondecreasing

↪ $\min(\varepsilon, \varepsilon)$ concave, nondecreasing

↪ $1 - \log(\varepsilon)$ convex, nonincreasing

Expression Tree for DCP Verification (Fig 14.2)



Conditions Discovered

- Require $x_2 > 0$ for $\log(x_2)$
- Require $\min(x_1, x_2) \geq 0$ for $\sqrt{\cdot}$
- Hence need $x_1 \geq 0$ and $x_2 > 0$

Canonicalization — What and Why?

Definition

Canonicalization is the process of converting a DCP into *partitioned canonical form* that existing solvers can directly handle. It enables dual information (lower bounds, dual certificates, convergence assessment).

Partitioned Canonical Form (Eq. 14.8)

$$\begin{aligned} & \underset{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}}{\text{minimize}} && d + \sum_{i=1}^k (\mathbf{c}^{(i)})^\top \mathbf{x}^{(i)} \\ & \text{subject to} && \mathbf{x}^{(i)} \in S^{(i)} \quad (i = 1:k), \quad \sum_{i=1}^k \mathbf{A}^{(i)} \mathbf{x}^{(i)} = \mathbf{b} \end{aligned}$$

Each $S^{(i)}$ is a convex set from the atom library. Coupling happens only through $\mathbf{Ax} = \mathbf{b}$.

Two Steps in Canonicalization

• **Linearization** — introduce new atom variables

Linearization

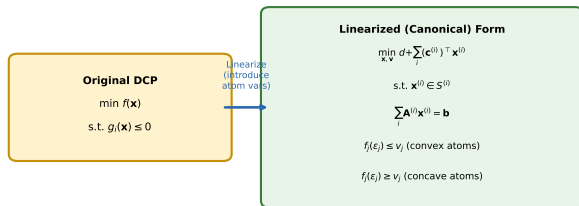
Process (Eqs. 14.9–14.11)

Given a product-free expression $a + \mathbf{b}^\top \mathbf{x} + \sum_{j=1}^q c_j f_j(\mathbf{e}_j)$, the **linearized** form introduces atom variables $\mathbf{v}$:

$$a + \mathbf{b}^\top \mathbf{x} + \mathbf{c}^\top \mathbf{v} \quad \text{with} \quad \begin{cases} f_j(\mathbf{e}_j) \leq v_j & \text{if } f_j \text{ is convex} \\ f_j(\mathbf{e}_j) \geq v_j & \text{if } f_j \text{ is concave} \end{cases}$$

Introduces q new variables and q new inequality constraints.

Canonicalization: Linearization Step



Key idea

Each atom call $f_j(\cdot)$ is replaced by a fresh variable v_j plus a *constraint* bounding v_j to the atom's value.

This makes all function arguments affine, which is what standard solvers require.

Linearization — Numerical Example (Ex. 14.8)

Linearizing: $\text{pow}(3 + 2 \log(1 + x_1) - \sin(x_2), -1)$

Step 1: Innermost arguments $1 + x_1$ and x_2 are affine — no action needed.

Step 2: Next level $3 + 2 \log(1 + x_1) - \sin(x_2)$ is not affine. Introduce $v_1, v_2, u_{1,1}$:

$$3 + 2v_1 - v_2 \quad \text{s.t.} \quad \log(u_{1,1}) \geq v_1, \quad u_{1,1} = 1 + x_1, \quad \sin(x_2) \leq v_2$$

Step 3: Top-level $\text{pow}(\cdot, -1)$ introduces $v_3, u_{3,1}$:

$$v_3 \quad \text{s.t.} \quad \text{pow}(u_{3,1}, -1) \leq v_3, \quad u_{3,1} = 3 + 2v_1 - v_2$$

Result: 5 new variables, 5 new constraints — *all functions now applied to affine arguments.*

Graph Implementations — Absolute Value

Graph Expansion: $|x|$ (Eq. 14.13)

The atom $|x|$ is nonlinear but can be expressed as the solution to a **linear program**:

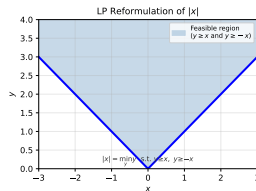
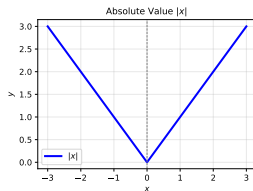
$$|x| \Rightarrow \begin{cases} \min_y y \\ \text{s.t. } y \geq x, \quad y \geq -x \end{cases}$$

This *graph implementation* removes the nondifferentiable kink at $x = 0$.

Usage: constraint $|2x + 3| \leq 5$

Original: $v = |u|$, $u = 2x + 3$, $v + s = 5$, $s \geq 0$

Expanded: $v \geq u$, $v \geq -u$, $u = 2x + 3$, $v + s = 5$, $s \geq 0$



Epigraph and Hypograph

Definitions (Eq. 14.15)

$$\text{epi } f = \{(\mathbf{x}, y) \in \mathbb{R}^n \times \mathbb{R} \mid y \geq f(\mathbf{x})\}$$

$$\text{hypo } f = \{(\mathbf{x}, y) \in \mathbb{R}^n \times \mathbb{R} \mid y \leq f(\mathbf{x})\}$$

- f is convex $\iff$ $\text{epi } f$ is a convex set
- f is concave $\iff$ $\text{hypo } f$ is a convex set

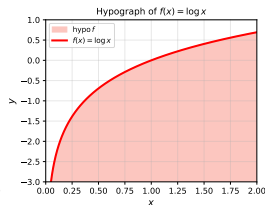
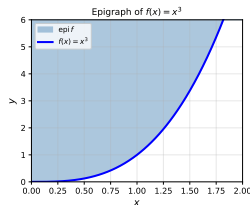
Graph Expansion via Epigraph/Hypograph

Replace convex atom constraint:

$$f(\mathbf{x}) \leq y \iff (\mathbf{x}, y) \in \text{epi } f$$

Replace concave atom constraint:

$$f(\mathbf{x}) \geq y \iff (\mathbf{x}, y) \in \text{hypo } f$$



Blue: epigraph of x^3 (convex). Red: hypograph of $\log x$ (concave).

Graph Implementations — Norms (Ex. 14.9)

L_1 and L_∞ Norms as LPs

$$\|\mathbf{x}\|_1 \Rightarrow \begin{cases} \min_{\mathbf{v}} \mathbf{1}^\top \mathbf{v} \\ \text{s.t. } -\mathbf{v} \leq \mathbf{x} \leq \mathbf{v} \end{cases}$$

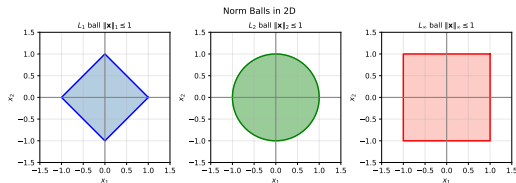
$$\|\mathbf{x}\|_\infty \Rightarrow \begin{cases} \min_{\mathbf{v}} v \\ \text{s.t. } -v\mathbf{1} \leq \mathbf{x} \leq v\mathbf{1} \end{cases}$$

General L_p Norm ($p \geq 1$, Eq. 14.18)

$$\|\mathbf{x}\|_p \Rightarrow \begin{cases} \min_{\mathbf{v}} \mathbf{1}^\top \mathbf{v} \\ \text{s.t. } |x_i|^p \leq v_i \quad \forall i \end{cases}$$

Solved via interior point methods.

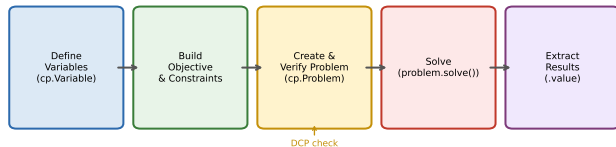
L_2 norm as Least Squares (Ex. 14.10)



Unit balls for L_1 , L_2 , L_∞ norms in 2D. Each shape encodes the solver-compatible form.

CVXPY Overview

CVXPY Workflow for Disciplined Convex Programming



Python equivalent of Convex.jl

The book uses Convex.jl (Julia).
Python users should use **CVXPY**.

Install: `pip install cvxpy`

What CVXPY does

- 1 Parses the problem into an expression graph
- 2 Runs automatic DCP verification
- 3 Performs canonicalization (linearize + graph expand)
- 4 Calls an appropriate solver (SCS, ECOS, MOSEK, ...)
- 5 Returns solution values

Supported problem types

- LP (Linear Programs)
- QP / SOCP (Second-order cone)
- SDP (Semidefinite programs)
- GP (Geometric programs)
- MIP (Mixed-integer, partial)

CVXPY: Basic Usage

```
import cvxpy as cp
import numpy as np

# Define matrices
A = np.array([[2, -1], [1, 3]])
b = np.array([4, -5])
C = np.array([[0, 1], [3, -4]])
d = np.array([-1, 0])

# Decision variable
x = cp.Variable(2)

# Objective and constraints
objective = cp.Minimize(cp.norm(A @ x - b, 1))
constraints = [cp.norm(C @ x - d, 1.5) <= 3]

# Form and solve
problem = cp.Problem(objective, constraints)
problem.solve()

print("Status: ", problem.status)
print("Optimal x:", x.value)
print("Obj value:", problem.value)
```

Listing 1: CVXPY basics — Python equivalent of Convex.jl Ex. 14.1

CVXPY: Checking DCP Compliance

```
import cvxpy as cp
import numpy as np

x = cp.Variable(2)
A = np.array([[2, -1], [1, 3]])
b = np.array([4., -5.])
C = np.array([[0., 1.], [3., -4.]])
d = np.array([-1., 0.])

objective = cp.Minimize(cp.norm(A @ x - b, 1))
constraint = cp.norm(C @ x - d, 1.5) <= 3
problem = cp.Problem(objective, [constraint])

# Check DCP compliance BEFORE solving
print("Is DCP? ", problem.is_dcp()) # True
print("Obj DCP? ", objective.expr.is_convex()) # True
print("Constr? ", constraint.is_dcp()) # True
print("n_vars: ", problem.variables())
print("n_constr:", len(problem.constraints))

problem.solve(solver=cp.ECOS)
print(f"Optimal: {x.value}")
```

Listing 2: Verify DCP status programmatically

CVXPY: Lasso Regression

```
import cvxpy as cp
import numpy as np

np.random.seed(42)
n, m = 100, 30          # n samples, m features
A = np.random.randn(n, m)
x_true = np.random.randn(m) * (np.random.rand(m) > 0.7)
b = A @ x_true + 0.1 * np.random.randn(n)

lam = 0.5                # regularisation strength
x = cp.Variable(m)

objective = cp.Minimize(
    cp.sum_squares(A @ x - b) + lam * cp.norm1(x)
)
problem = cp.Problem(objective)
print("Is DCP?:", problem.is_dcp())  # True
problem.solve()

print(f"Recovered {np.sum(np.abs(x.value) > 0.01)} nonzeros")
print(f"True nonzeros: {np.sum(np.abs(x_true) > 0)}")
print(f"Residual: {np.linalg.norm(A @ x.value - b):.4f}")
```

Listing 3: Lasso: L1-regularized least squares with CVXPY

CVXPY: Portfolio Optimization

```
import cvxpy as cp
import numpy as np

np.random.seed(0)
n = 10                                # number of assets
mu = np.random.randn(n) * 0.1 + 0.05  # expected returns
Sig = np.random.randn(n, n)
Sig = Sig.T @ Sig / n + 0.01 * np.eye(n)  # covariance matrix
gamma = 1.0                            # risk-aversion parameter

w = cp.Variable(n)                    # portfolio weights

# DCP: minimize risk - gamma * return
risk = cp.quad_form(w, Sig)           # convex (PSD matrix)
ret = mu @ w                          # affine
objective = cp.Minimize(risk - gamma * ret)
constraints = [cp.sum(w) == 1,        # budget constraint
              w >= 0]                 # long-only

problem = cp.Problem(objective, constraints)
print("Is DCP?", problem.is_dcp())    # True
problem.solve()

print(f"Optimal weights: {np.round(w.value, 3)}")
print(f"Expected return: {(mu @ w.value):.4f}")
print(f"Portfolio risk: {float(cp.sqrt(risk).value):.4f}")
```


CVXPY: Solving via Interior Point Methods

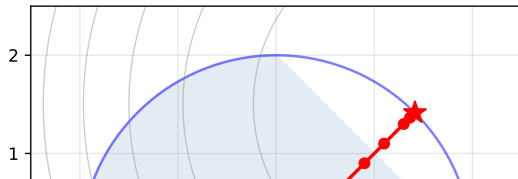
Interior Point Method for Canonical Form

Canonical form enables efficient interior point (barrier) methods. For the partitioned problem, the Hessian is block-diagonal:

$$\nabla^2 p_{\text{barrier}}(\mathbf{x}) = \begin{bmatrix} \nabla^2 p_{\text{barrier}}^{(1)}(\mathbf{x}^{(1)}) & & \\ & \ddots & \\ & & \nabla^2 p_{\text{barrier}}^{(k)}(\mathbf{x}^{(k)}) \end{bmatrix}$$

Block-diagonal inverse is *extremely fast* to compute.

Interior Point Method on Convex QP



```
1 import cvxpy as cp
2 import numpy as np
3
4 x = cp.Variable(2)
5 A = np.eye(2)
6 b = np.array([1., 2.])
7
8 prob = cp.Problem(
9     cp.Minimize(cp.norm(A @ x - b, 2)),
10    [cp.norm(x, 1) <= 2.0]
11 )
12
13 # Choose specific solver
14 prob.solve(solver=cp.SCS)           # SCS
15 prob.solve(solver=cp.ECOS)         # ECOS (IP)
16 # prob.solve(solver=cp.MOSEK)      # MOSEK (
17     commercial)
```

Chapter Summary

Key Takeaways

- 1 **DCP** is a framework for writing convex optimization problems in a form that automated tools can verify and solve.
- 2 **Verification** proceeds in two stages: (1) product-free check, then (2) sign/composition/top-level rules applied recursively down the expression tree.
- 3 **Canonicalization** transforms a DCP into partitioned canonical form via linearization and graph implementations.
- 4 **Interior point methods** efficiently solve canonical-form problems by exploiting block-diagonal Hessian structure.
- 5 **CVXPY** (Python) / **Convex.jl** (Julia) implement DCP: parse, verify, canonicalize, and call a numerical solver.

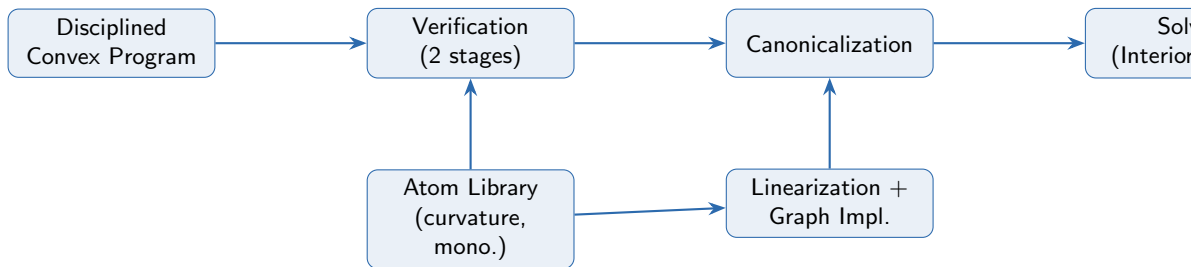
Ecosystem

Python:
CVXPY

Julia:
Convex.jl

Reference:
dcp.stanford.edu

Concept Map



Selected Exercises I

Exercise 14.1 — Extended-Value Functions

Show that if f is convex over a convex set $\mathcal{S}$, then $g(\mathbf{x}) = f(\mathbf{x})$ if $\mathbf{x} \in \mathcal{S}$, ∞ otherwise, is also convex over $\mathbb{R}^n$.

Hint: Check the three cases: both $\mathbf{x}, \mathbf{y} \in \mathcal{S}$; exactly one in $\mathcal{S}$; neither in $\mathcal{S}$.

Exercise 14.2 — Squares in DCP

Products are not DCP-compliant, but x^2 can be incorporated by defining a `square(x)` atom with: curvature = convex, nondecreasing for $x \geq 0$, nonincreasing for $x \leq 0$, range = $[0, \infty)$.

Selected Exercises II

Exercise 14.5 — Graph Implementation of $\sqrt{x}$

$\sqrt{x}$ is concave, so use its hypograph:

$$\text{hypo}\sqrt{x} = \{(x, y) \mid x \geq 0, y \leq \sqrt{x}\} = \{(x, y) \mid x \geq 0, y^2 \leq x\}$$

Graph implementation: $\sqrt{x} \Rightarrow \arg \max_y y$ s.t. $y^2 \leq x$

This is differentiable everywhere.

Exercise 14.6 — Canonicalize a Norm Problem

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_1 \quad \text{s.t.} \quad \|\mathbf{Cx} - \mathbf{d}\|_{1.5} \leq 3$$

Apply L_1 graph expansion: $-\mathbf{y}^{(1)} \leq \mathbf{Ax} - \mathbf{b} \leq \mathbf{y}^{(1)}$. Apply L_p expansion: $|(\mathbf{Cx} - \mathbf{d})_i|^{1.5} \leq y_i^{(2)}$, minimize $\mathbf{1}^\top \mathbf{y}^{(2)} + s = 3$.

CVXPY: Solving Exercise 14.6 Numerically

```
import cvxpy as cp
import numpy as np

A = np.array([[2., -1.], [1., 3.]])
b = np.array([4., -5.])
C = np.array([[0., 1.], [3., -4.]])
d = np.array([-1., 0.])

x = cp.Variable(2)

objective = cp.Minimize(cp.norm(A @ x - b, 1))
constraints = [cp.norm(C @ x - d, 1.5) <= 3]

problem = cp.Problem(objective, constraints)
print("Is DCP:", problem.is_dcp()) # True

problem.solve(solver=cp.ECOS, verbose=False)
print(f"Status : {problem.status}")
print(f"Optimal : x = {x.value}")
print(f"Obj val : {problem.value:.6f}")
print(f"||Cx-d||_1.5 = {float(cp.norm(C@x-d, 1.5).value):.4f}")
```

Listing 6: Numerical solution of the mixed-norm problem (Ex. 14.6)

Output (approximate)

Further Reading

Primary References

- M. Grant, S. Boyd, Y. Ye, “*Disciplined Convex Programming*,” in *Global Optimization: From Theory to Implementation*, pp. 155–210, 2006.
- S. Diamond, S. Boyd, “*CVXPY: A Python-Embedded Modeling Language for Convex Optimization*,” *JMLR*, 2016.
- S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. (Free at cvxbook.stanford.edu)

Online Resources

- dcp.stanford.edu — interactive DCP sandbox
- cvxpy.org — CVXPY documentation and examples
- cvxgrp.stanford.edu — Stanford Convex Optimization Group